Real-World Data Preparation with Python

> *“Nobody's model was wrong. The data just wasn't telling the truth we thought it was.”*

Zipporah Muriithi • Junior Data Analyst & Scientist • 2026

01 / FOUNDATIONS

What Is Data Preparation, Really?

🥕🔪🍲

Somebody had to write the data down — a clerk, a sensor, a form on a phone. That means it's never "the truth." It's a **recording** of the truth, made under someone else's conditions.

Data preparation is what I actually spend most of my week doing: taking data collected for one reason and making it trustworthy for a completely different one.

It's the vegetables-before-cooking part. Nobody claps for it. Skip it and the plate still looks fine — you just can't trust what's on it.

TERM	WHAT I ACTUALLY MEAN BY IT
Raw data	Untouched. What arrived, nothing more
Clean data	Known problems fixed, or at least flagged
Tidy data	One row, one observation — no surprises
Missing data	Should be there. Isn't. Not the same as zero
Duplicate	One event, counted more than once by accident
Grain	The question I ask on every project: what is one row?

02 / STAKES

Why It Matters: The Cost of Getting It Wrong

WHAT WENT WRONG IN THE DATA	WHAT IT COST, IN THE REAL PROJECT
Duplicate orders in the feed	**Revenue booked twice; the forecast was garbage**
A missing reading saved as 0	**The model quietly learned the wrong pattern**
Tomorrow's data leaking into training	**Looked brilliant in testing, fell apart live**
Matching customers by name alone	**Same person split into three "different" people**

I've watched every one of these happen. Nothing crashed. Nobody got an error. The report just went out wrong, and everyone believed it.

03 / HISTORY

Who Built This Discipline

1662
Graunt tames London's messy death records — demography is born

1854
Nightingale's chart changes hospital policy overnight

1970
Codd gives us keys, tables, and joins

1991
Guido quietly releases Python

1999
CRISP-DM gives the industry a shared map

2008
McKinney builds pandas — his job needed it

None of this was one genius moment. It was generations of people getting tired of not trusting their own numbers.

04 / THE LIFECYCLE

Where Preparation Lives: CRISP-DM

Business Understanding → **Data Understanding** → **DATA PREPARATION** → **Modeling** → **Evaluation** → **Deployment**

↻ I've never once run this in a straight line. Something always sends you back to preparation.

Conference talks are mostly about modeling. My calendar is mostly about the box in teal.

05 / METHOD

How I Actually Work Through It

1. Ask what **one row** even means, before touching anything
2. Track down every source — and who owns it
3. Keep a copy of the mess, **untouched**, always
4. Look at it properly **before** I fix a thing
5. Write the rule down before I make an edit I can't undo
6. Clean it, join it, reshape it
7. Test it at every handoff point, not just at the end
8. Hand it over with a note explaining what I did and why

A Few Industries, A Few War Stories

INDUSTRY	THE CATCH NOBODY FLAGS AT FIRST	WHAT ACTUALLY GOES WRONG
Banking	Credit history has to reflect what was known that day, not today	You approve a loan someone can't repay
Retail	Is that "0 sales" no demand — or just an empty shelf?	You reorder the wrong stock, again
Healthcare	A blank field is not the same as a zero	A patient gets triaged wrong
Telecom	Are you predicting churn, or reacting to it?	You "save" customers who already left
Public health	Every facility counts things slightly differently	Medicine ends up in the wrong district

Why I Reach for Python First

- **Delivery — the dashboard, the API, the report someone reads**
- **Orchestration — Airflow, Dagster, Prefect**
- **Validation — Pandera, Great Expectations**
- **Processing — pandas, Polars, DuckDB, Spark**
- **Storage — wherever the data actually lives**

✓ I don't switch languages between the messy part and the shipped part

✓ pandas, Polars, DuckDB — I pick based on how big the file actually is

✓ A non-programmer can still read what the code is doing

✓ What I test in a notebook is what runs in production

I won't pretend it's the best tool for everything — SQL still beats it inside a warehouse, and I know good R statisticians who'd never switch. I use Python because it's the one language that follows the data the whole way.

"Looks Clean To Me" Isn't a Standard

```
rules:
  - customer_id is never null
  - status is one of [active, suspended, closed]
  - freshness ≤ 26 hours
```

Privacy
I ask what I actually need before I ask what I can get. Fewer fields, fewer ways to get someone hurt.

Bias
I go looking for it in the data before the model ever sees it — by then it's usually too late to fix cheaply.

What I'd Want You to Remember Monday

1. Whatever you're staring at is a recording, not the truth. Ask who made it and why.
2. "Clean" only means something once you say clean *for what*.
3. This unglamorous stage is where most of your real risk actually sits.
4. Write your rules down. Keep the raw copy. Don't skip this because you're in a hurry.

→ **Go find the one spreadsheet at work you already don't trust. Start there.**